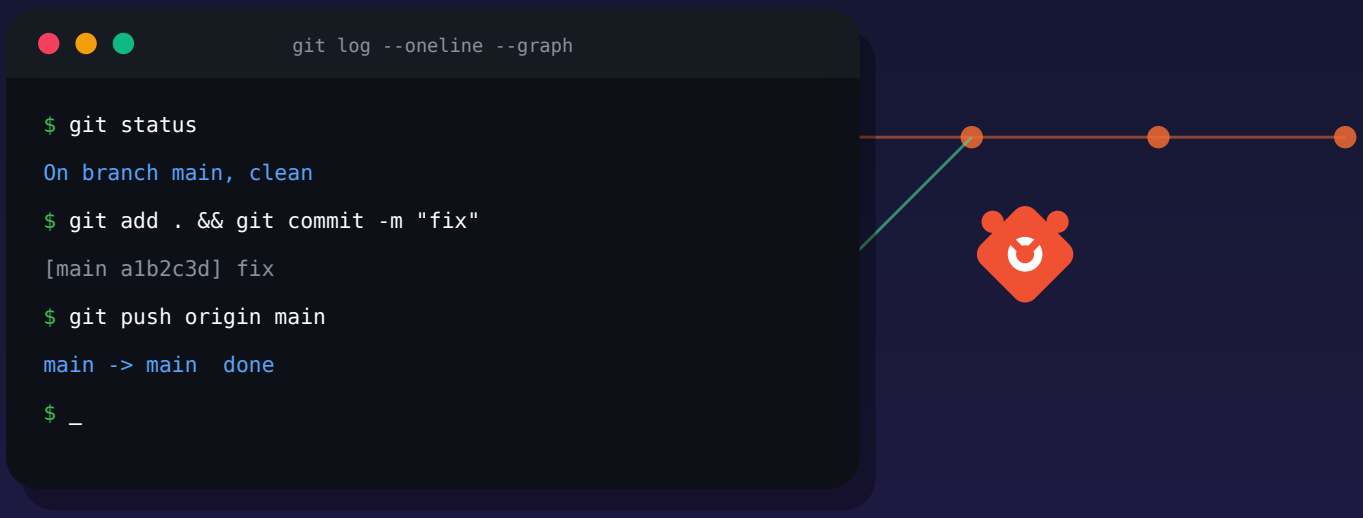
VISUAL STUDY NOTES



Git Notes

Every Git concept and command, start to finish.

Beginner-friendly. Straight to the point. A full command reference.

CREATED BY

Sharad Verma @sharadverma638

Branching | Merging | Rebase | Stash | Undo | Remotes

---

17 sections

25+ illustrations

100+ commands

Want to learn DevOps?

github.com/sharadverma638/sv-opslibrary

Edition 2026 | Companion guide: GitHub Notes | by Sharad Verma

Contents

17 sections, start to finish. Follow in order, or jump to what you need.

01 **What Is Git?** Version control, in plain language 2

02 **Installing & Configuring Git** Every OS, plus your identity 4

03 **How Git Works** The four areas every change moves through 6

04 **Your First Repository** init, clone, status, and structure 7

05 **The Core Workflow** status, add, commit, log, diff 9

06 **Ignoring Files** .gitignore and what never belongs in Git 11

07 **Branching** Work on features without breaking main 13

08 **Merging & Conflicts** Bringing branches back together 14

09 **Git Stash** Park unfinished work safely 16

10 **Undoing Mistakes** restore, reset, and revert 17

11 **Git Rebase** A cleaner, straighter history 19

12 **Git History & Inspection** log, show, blame, bisect, reflog 21

13 **Git Tags** Marking releases forever 24

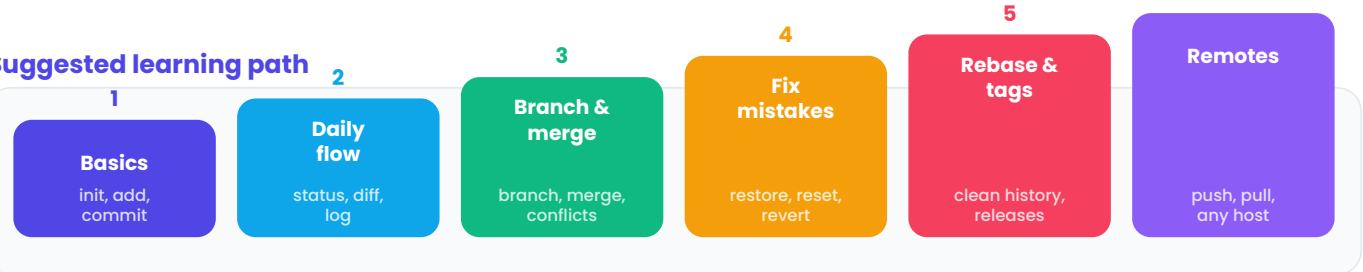
14 **Remotes** Connecting to GitHub or any host 25

15 **Advanced Git Toolkit** cherry-pick, worktree, hooks, submodules 27

16 **Best Practices** Habits that prevent Git headaches 30

17 **Cheat Sheet & Glossary** Every command, one place 32

Suggested learning path



Build these skills in order; each stage uses the one before it.

01 What Is Git?

Version control, in plain language

Git is a **version control system**: a tool that tracks changes to files over time. It lets you save progress, return to earlier versions, and work without ever losing important edits.

Think of Git as a **timeline for your project**. Each saved point is called a **commit**. You can review changes, compare versions, and undo mistakes whenever you need to.

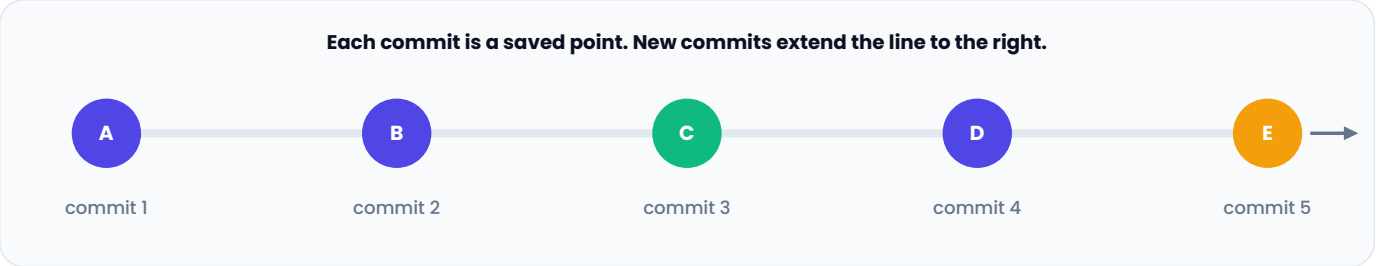


Fig 1. A commit timeline: each letter is one saved point.

Git is most often used together with **GitHub**, a website that gives you a place to store repositories online, share code, and collaborate with others. (GitHub gets its own notes; this guide is about Git, the tool itself.)

Why use Git?

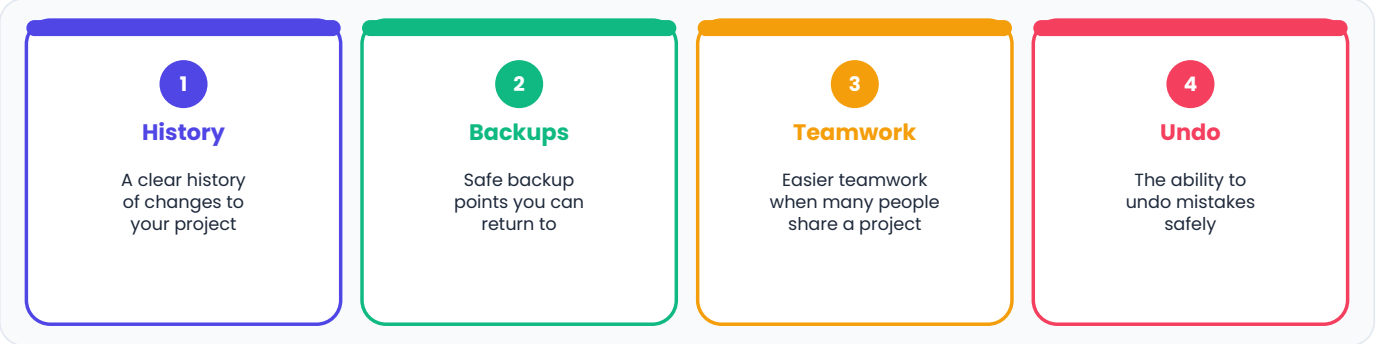


Fig 2. Four reasons people reach for Git.

A simple workflow

1. Edit your files
2. Save the changes
3. Commit them
4. Push them to a remote host (like GitHub)

A brief history of Git

Git was created by **Linus Torvalds in 2005** while he was developing the Linux kernel. He needed a fast, reliable tool to manage changes across a huge team of contributors. Before Git, developers often used older systems that were slower or harder to work with; Git made branching and merging simple and efficient. Its design focused on **speed, data integrity, and distributed work** — which is why it became the standard tool for modern software development.

Why Git became popular

- It works well for large projects with many contributors.
- It supports branching without much overhead.
- It is fast and reliable, even on large codebases.
- It is free and open source.

Understanding version control systems

A **version control system (VCS)** tracks file changes over time. It lets you save snapshots, compare versions, and collaborate without overwriting each other's work. There are two broad models.

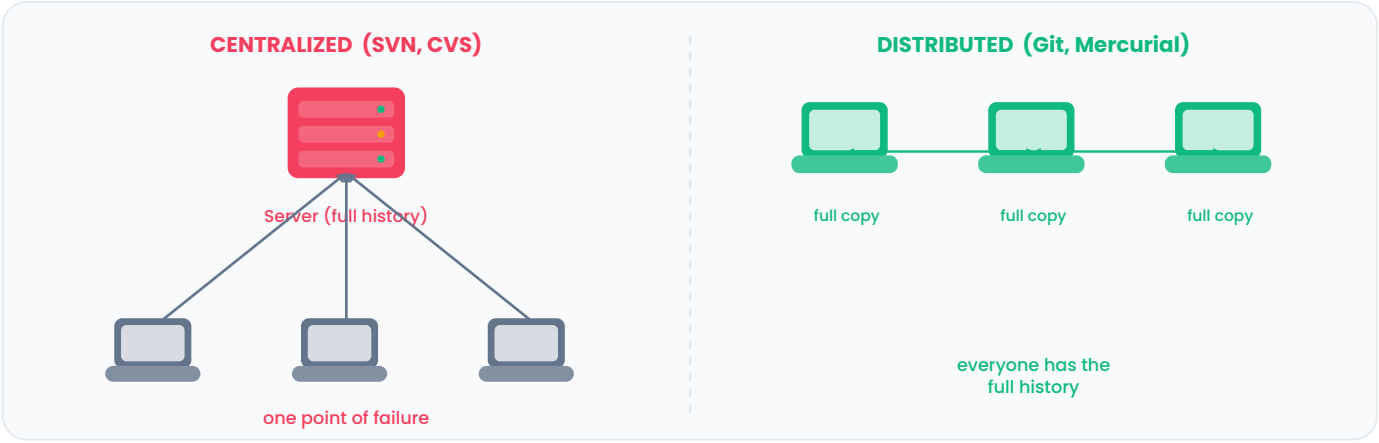


Fig 3. Centralized: one server holds history. Distributed: everyone has the full history.

	Centralized (SVN, CVS)	Distributed (Git, Mercurial)
Full history	On the server only	On every machine
Offline work	Limited	Full commits, locally
Backup	Single point of failure	Many copies already exist

KEY IDEA

In a **centralized** model, one server holds the official copy; developers check files out, change them, and check them back in. If the server goes down, collaboration stops. In a **distributed** model, every developer has a full copy of the project history on their own machine. You commit locally first, then sync with a remote when ready — so you can work fully offline.

02 Installing & Configuring Git

Every OS, plus your identity

Install Git once, then use it from the terminal, VS Code, or GitHub Desktop. The recommended approach is to download it from the official site: **git-scm.com**.

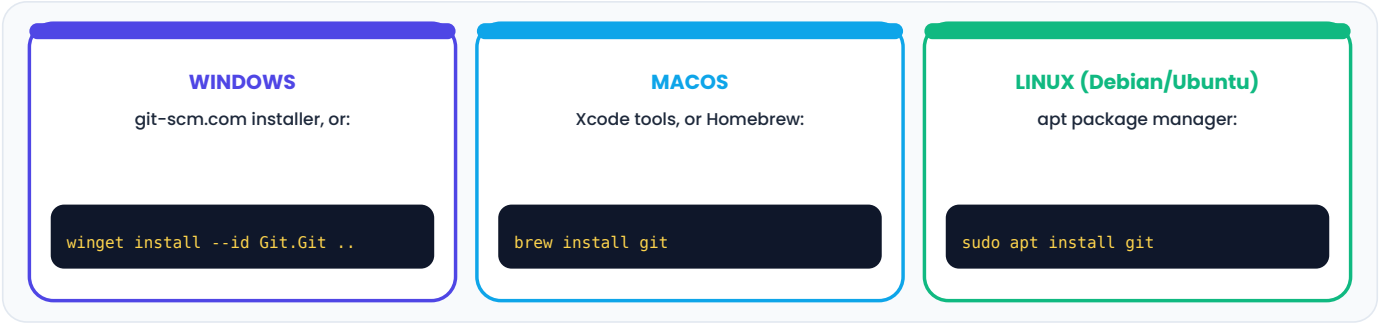


Fig 4. Installing Git on Windows, macOS and Linux.

Windows

```
winget install --id Git.Git -e --source winget
```

Or download the installer from git-scm.com and run it with default options.

macOS

```
xcode-select --install
# or
brew install git
```

Linux (Debian / Ubuntu)

```
sudo apt update
sudo apt install git
```

Verifying installation

```
git --version
```

You should see something like `git version 2.x.x`.

Choosing your workflow

Way of using Git	Good for
Command Line	Full control, works everywhere. The most powerful and portable way to use Git.
VS Code	Built-in Source Control panel, good for daily edits alongside your code.
GitHub Desktop	Visual interface, great for beginners who prefer a graphical experience.

All three use the exact same Git commands under the hood.

Configuring Git

Set your identity once so every commit is attributed to you.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Use the same email as your GitHub account so commits link back to your profile.

Viewing configuration

```
git config --list            # all settings
git config user.name          # one value
git config user.email
```

Global vs local configuration



Fig 5. Narrower settings override wider ones.

Scope	Flag	Applies to
Global	`--global`	All repos on your machine. Use this for your usual name and email.
Local	(no flag, run inside a repo)	One project only. Useful for overriding your email for a work or client repo.

Overriding config for one repo:

```
cd my-project
git config user.email "work@company.com"
```

Useful defaults

What it sets	Command
Default branch name	`git config --global init.defaultBranch main`
Colored output	`git config --global color.ui auto`
Default editor	`git config --global core.editor code`

03 How Git Works

The four areas every change moves through

Git moves your changes through **four areas** before they reach a remote host like GitHub.

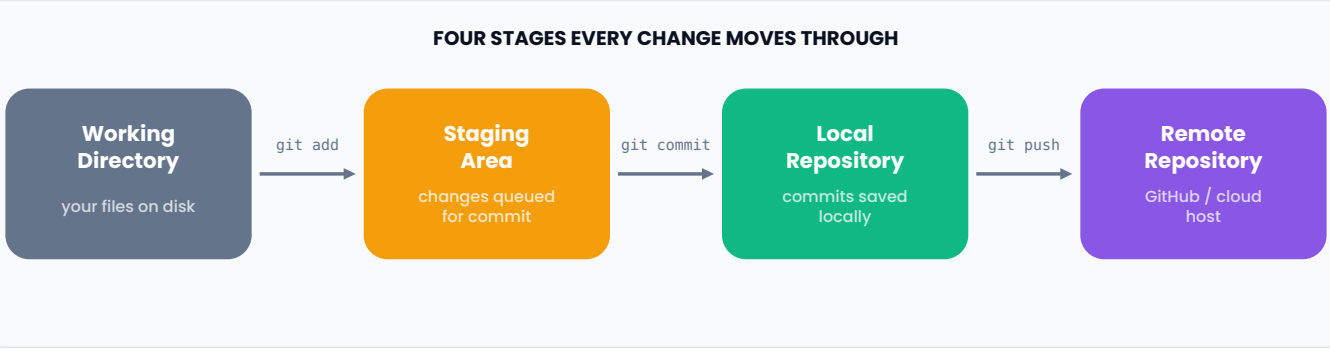


Fig 6. Working directory -> staging area -> local repository -> remote repository.

Area	What it is
Working Directory	Your project files on disk. Edit freely here. Git sees changes but does not save them until you stage and commit.
Staging Area	A holding zone for changes you want in the next commit. Use `git add` to move files here.
Local Repository	Where commits are stored permanently on your machine. Use `git commit` to save a snapshot.
Remote Repository	The copy on GitHub (or another host). Use `git push` to upload and `git pull` to download.

The full Git workflow

```
# 1. Edit files in your working directory

# 2. Stage changes
git add .

# 3. Commit to local repository
git commit -m "describe your change"

# 4. Push to remote repository
git push origin main
```

Understanding snapshots

Each commit is a **snapshot of your entire project** at one moment in time. Git stores these efficiently, so you can jump back to any point in history.

```
git log --oneline   # see all snapshots
```

04 Your First Repository

init, clone, status, and structure

A Git repository starts with one command in your project folder.

```
mkdir my-project
cd my-project
git init
```

This creates a hidden `.git` folder that stores all history.

git init

Run `git init` in any folder to start tracking it. Only run it once per project.

```
git init                 # start tracking the current folder
git init my-new-project    # start tracking a new folder
```

git clone

If the repository already exists on a remote host, use `git clone` to download a complete copy to your local machine.

```
git clone <repository-url>
```

Understanding repository structure

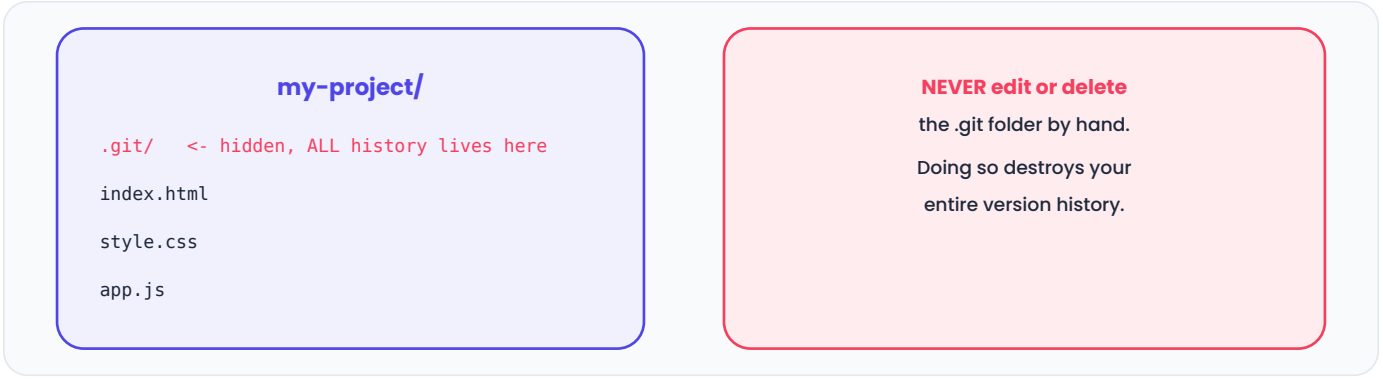


Fig 7. The hidden .git folder holds everything.

WATCH OUT
Never delete or edit the .git folder manually. Doing so destroys the entire version history of your project.

Checking repository status

```
git status
git status -s   # short, two-column summary
```

Shows which files are new, modified, or staged. Run this often.

Reading git status output

```
On branch main
Changes to be committed:
   new file:   index.html

Changes not staged for commit:
   modified:   style.css

Untracked files:
   app.js
```

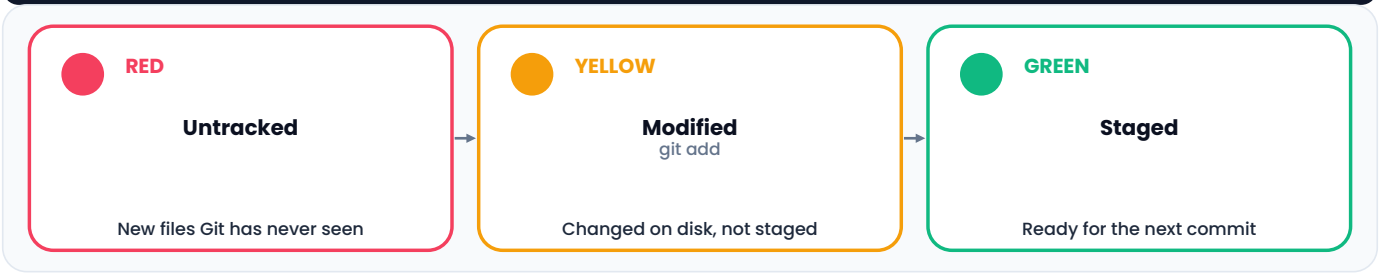


Fig 8. Red = untracked, yellow = modified, green = staged.

05 The Core Workflow

status, add, commit, log, diff

These commands cover most of your daily Git work.

git status

```
git status
git status -s
```

git add — stages changes for the next commit

```
git add README.md   # stage one file
git add .            # stage all changes
git add *.py         # stage all files of one type
git add -p           # stage interactively, hunk by hunk
```

git commit — saves a snapshot with a message

```
git commit -m "add readme file"

# stage and commit tracked files in one step
git commit -am "fix typo in main script"
```

Note: `git commit -am` only works for files Git already tracks — it will not pick up brand-new untracked files.

git log — shows commit history

```
git log
git log --oneline            # compact view
git log --oneline -5         # last 5 commits
git log --oneline --graph --all   # graph view
```

The Git lifecycle

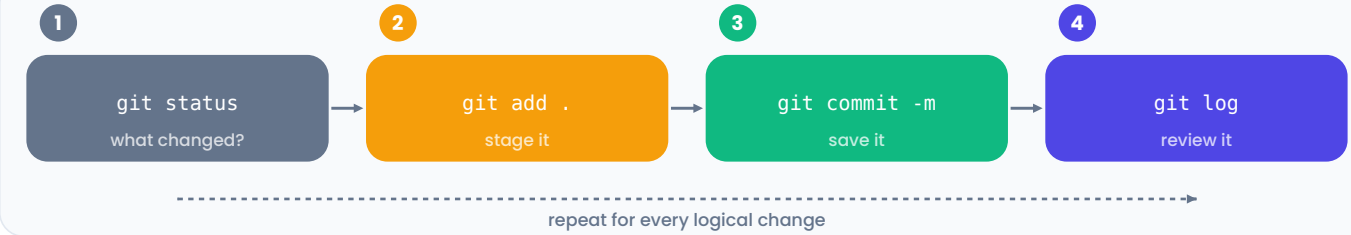


Fig 9. Repeat this loop for every logical change.

```
git status                # what changed?
git add .                 # stage it
git commit -m "msg"       # save it
git log --oneline         # review it
```

git diff: seeing exactly what changed

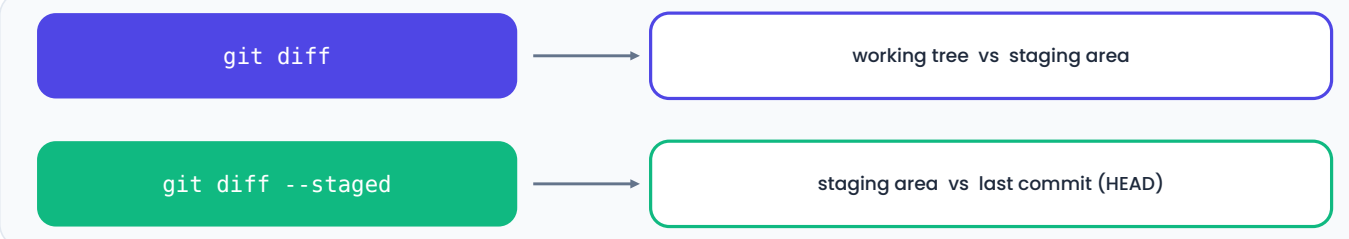


Fig 10. git diff compares different pairs of areas.

```
git diff            # unstaged changes (working tree vs staging)
git diff --staged   # staged changes (staging vs last commit)
git show <commit>   # see the details of a specific commit
```

Best practices while committing

- Commit when a logical piece of work is done.
- One idea per commit.
- Use present-tense messages: `add login form`, not `added login form`.

Command reference at a glance

Command	What it does
git status	Shows staged, modified, and untracked files
git status -s	Compact two-column summary
git add .	Stages all changes for the next commit
git commit -m	Creates a new snapshot with a message
git commit -am	Stages tracked files and commits in one step
git log --oneline	Compact one-line summary of each commit
git log --graph --all	Visualises the full branch and merge history

06 Ignoring Files

.gitignore and what never belongs in Git

Not every file belongs in Git. Some should stay local to your machine.

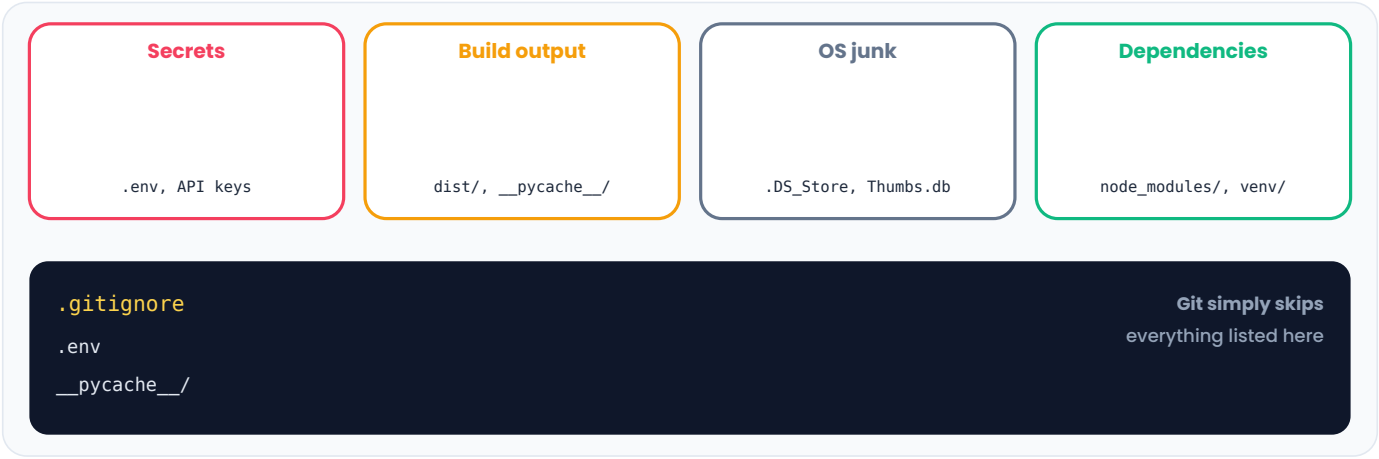


Fig 11. Create a .gitignore file; Git skips anything listed there.

Creating .gitignore

```
touch .gitignore
```

Example .gitignore

```
# environment variables
.env

# Python
__pycache__/
*.pyc
venv/

# OS files
.DS_Store
Thumbs.db

# build output
dist/
build/
```

Common .gitignore rules

```
secret.txt          # ignore a specific file
node_modules/        # ignore a folder and everything inside
*.log                # ignore all files of a type
!important.log       # but keep one specific file
```

We read `!important.log` as "not important.log", so it will not be ignored. `!` is the not operator.

Pattern	What it ignores
secret.txt	A specific file named secret.txt anywhere in the repo.
node_modules/	An entire folder and all of its contents, recursively.
***.log**	All files with the .log extension.

Pattern	What it ignores
!important.log	Exception: keep this file even if *.log is ignored above it.
.env	Environment variable file with secrets and API keys.

Empty folders and .gitkeep

Git does not track empty folders. To keep one, add a placeholder file:

```
mkdir logs
touch logs/.gitkeep
git add logs/.gitkeep
```

Un-tracking a file already in Git

If a file was already tracked before you added it to .gitignore:

```
git rm --cached filename
git commit -m "stop tracking filename"
```

07 Branching

Work on features without breaking main

Branches let you work on features without touching stable code.

What is a branch?

A branch is an **independent line of development**. `main` is the default branch name.

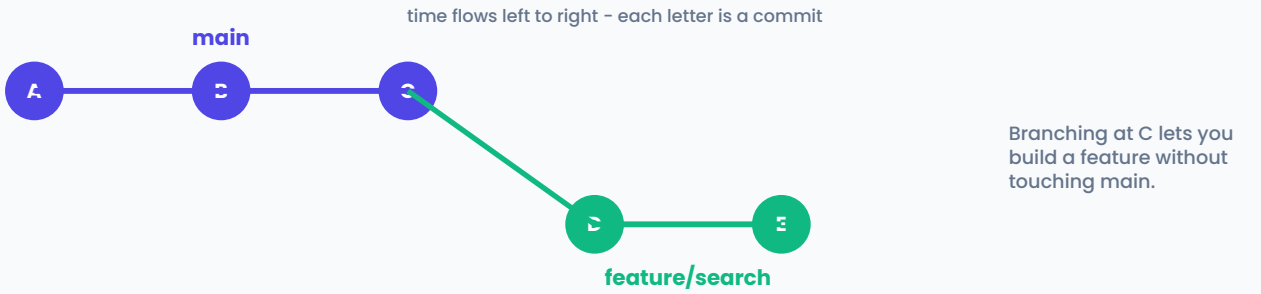


Fig 12. Branching at C lets you build a feature without touching main.

Why branches exist

- They help build features in isolation.
- They help fix bugs without breaking production code.
- They help you experiment safely.

Creating, switching, and both at once

`git branch name`	create a branch (stay where you are)
`git checkout name`	switch to a branch
`git switch name`	switch (modern alternative)
`git checkout -b name`	create AND switch in one step
`git switch -c name`	create AND switch (modern)

Fig 13. Five commands cover almost every branch move.

```
git branch feature/search          # create (stay where you are)
git checkout feature/search         # switch
git switch feature/search           # switch (modern alternative)

git checkout -b feature/search      # create AND switch
git switch -c feature/search        # create AND switch (modern)
```

Viewing branches, and the graph

```
git branch                  # local branches (* shows current)
git branch -a                # all branches, including remote
git branch --show-current    # current branch name only

git log --oneline --graph --all   # visualise splits and merges
```

08 Merging & Conflicts

Bringing branches back together

Merging combines the work from one branch into another.

What is merge?

Merge takes commits from a feature branch and adds them to your target branch (usually main).

```
git switch main            # switch to the branch you want to merge INTO
git merge feature/search   # bring in the feature branch
```

Fast-forward merge

Happens when main has no new commits since the branch was created. Git simply moves the pointer forward — no merge commit needed.



Fig 14. A fast-forward merge.

```
git switch main
git merge feature/search
# output: Fast-forward
```

Merge commit

Happens when both branches have new commits. Git creates a **merge commit** to combine them — it has two parents, so history keeps the fork visible.

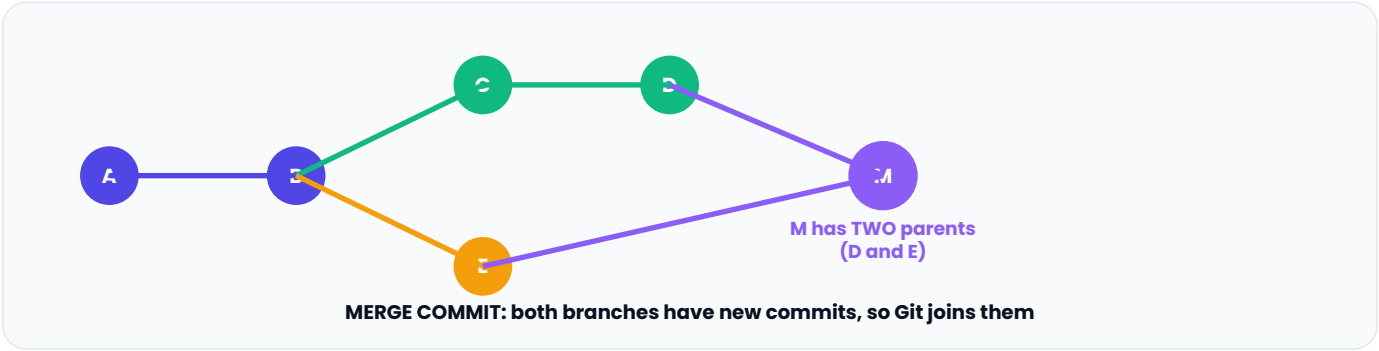


Fig 15. M has two parents: D and E.

```
git switch main
git merge feature/search
# output: Merge made by the 'ort' strategy
```

Deleting branches

After a successful merge, delete the feature branch:

```
git branch -d feature/search             # delete local branch
git push origin --delete feature/search   # delete remote branch
```

KEY IDEA
Deleted branches are still safe. The commits live in Git history: run `git log --oneline --all` to find them again.

Merge conflicts

Conflicts happen when Git cannot automatically combine two changes to the same lines.

Common causes:

- Two people edit the same line.
- You edit a file locally while someone else pushes changes to it.
- You merge or rebase branches with overlapping edits.

```
git merge feature/search
# output: CONFLICT in filename.py
```

Conflict markers

```
<<<<<<< HEAD
your version
=======
their version
>>>>>>> feature/search
```

Fig 16. What a conflict looks like inside the file.

Edit the file to keep the correct code, remove the markers, then:

```
git add filename.py
git commit -m "resolve merge conflict in filename.py"
```

Understanding git diff during a conflict

Command	Compares
`git diff`	working tree vs staging area
`git diff --staged`	staging area vs last commit (HEAD)
`git diff filename.py`	see conflict markers in one file

Best practices to avoid conflicts

```
git pull origin main       # pull before starting work
# ... do your work ...
git pull origin main       # pull again before pushing
git push origin main
```

Also: keep branches short-lived, and communicate with teammates about shared files.

Abort a merge if you are stuck

```
git merge --abort
```

Undoes the in-progress merge and returns your files to the state before you ran git merge.

09 Git Stash

Park unfinished work safely

Stash saves uncommitted work temporarily so you can switch tasks.

Why git stash exists

You are mid-feature but need to fix an urgent bug on main. Stash lets you park your changes without committing half-finished work.

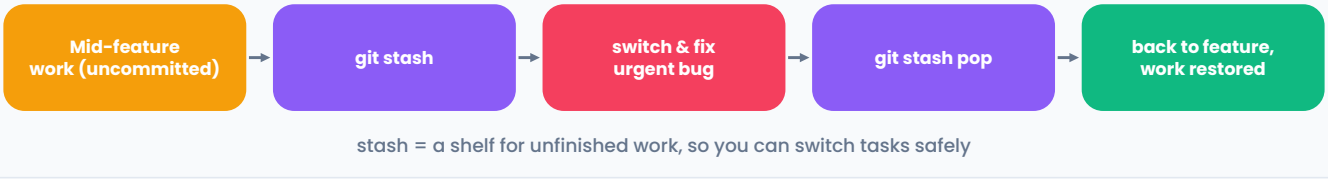


Fig 17. Stash, switch, fix, come back, restore.

Saving work temporarily

```
git stash

# with a label so you remember what it is
git stash push -m "wip on search feature"
```

Viewing and restoring stashes

```
git stash list

git stash apply             # apply most recent stash (keeps it in the list)
git stash apply stash@{1}   # apply a specific stash
```

Applying vs popping

Command	Effect
`git stash apply`	Applies the stash, but keeps it in the list.
`git stash pop`	Applies the stash AND removes it from the list.

Real-world example

```
git stash push -m "half-done login form"

git switch main
git switch -c hotfix/typo
# fix, commit, merge...

git switch feature/login
git stash pop
```

Cleaning up stashes

```
git stash drop stash@{0}   # delete a stash you no longer need
git stash clear             # clear all stashes
```

10 Undoing Mistakes

restore, reset, and revert

Git gives you several ways to undo work, depending on how far gone it is.



Fig 18. From safest to most destructive.

Undoing working directory changes

```
git restore filename.py   # discard edits to one file (not yet staged)
git restore .              # discard ALL unstaged edits
```

Undoing staged changes

```
git restore --staged filename.py   # unstage one file (keeps your edits)
git restore --staged .              # unstage everything
```

Git reset: moving the branch pointer

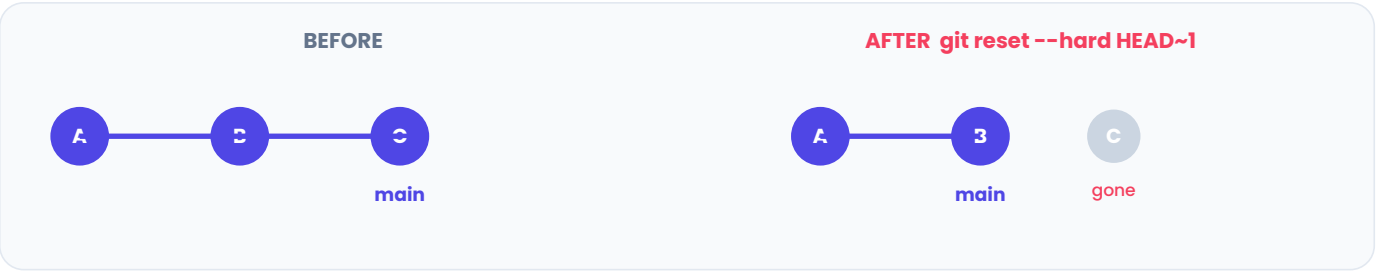


Fig 19. git reset --hard HEAD~1 moves main back and discards C.

```
git reset --soft HEAD~1   # undo last commit, KEEP changes staged
git reset --hard HEAD~1   # undo last commit, DISCARD everything
```

WATCH OUT
git reset --hard permanently deletes uncommitted work and the last commit. Use with care.

When NOT to use hard reset

- After you have already pushed to a shared remote (use git revert instead).
- When you are unsure what you will lose.
- On shared branches other people depend on.

The safer alternative: git revert

`git revert` creates a NEW commit that undoes a previous one, without rewriting history — safe for shared branches.

```
git revert HEAD
git revert a1b2c3d
```

C' reverses the changes from C. The original commit C stays in history and is safe for shared branches.

Rolling back: full reference

Command	What it does
git restore	Discard unstaged edits for a specific file.
git restore .	Discard all unstaged edits.
git restore --staged	Unstage a file (keeps edits).
git reset --soft HEAD~1	Undo last commit, keep file changes staged.
git reset --hard HEAD~1	Undo last commit, discard file changes entirely.
git revert	Safely undoes a commit that's already pushed.

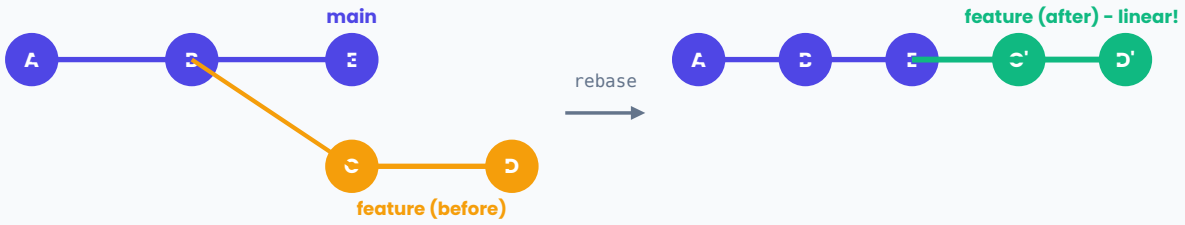
11 Git Rebase

A cleaner, straighter history

Rebase moves your branch commits on top of another branch for a cleaner, straighter history.

What is rebase?

Instead of creating a merge commit, rebase **replays** your commits on top of the latest main.



C' and D' are NEW commits with the same changes as C and D

Fig 20. C' and D' are new commits with the same changes as C and D.

```
git switch feature/search
git rebase main
```

Rebase vs merge

	MERGE	REBASE
Merge commit	Yes	No
History shape	Shows the fork	Straight line
Safe on shared branches	Yes	No – never rebase shared history

Fig 21. Merge preserves the fork. Rebase makes history linear.

When to use rebase

- Before opening a pull request, to clean up your branch.
- To sync a feature branch with an updated main.
- On branches only you work on.

```
# update your feature branch with the latest main
git switch feature/search
git fetch origin
git rebase origin/main
```

Best practices

```
git switch feature/search
git rebase main
git push --force-with-lease origin feature/search
```

Rules:

- Never rebase commits already pushed to a shared branch.
- If a rebase gets messy, abort and try again:

```
git rebase --abort
```

12 Git History & Inspection

log, show, blame, bisect, reflog

Your commit history is a searchable record of every change.

Commit IDs

Every commit gets a unique hash (SHA). You only need the first 7 characters most of the time.

```
git log --oneline
# a1b2c3d add login page
```

Viewing commit history

```
git log                          # full log
git log --oneline                # one line per commit
git log --stat                    # with file changes
git log --oneline -- README.md    # history for one file
```

Understanding commit graphs

```
git log --oneline --graph --all --decorate
```

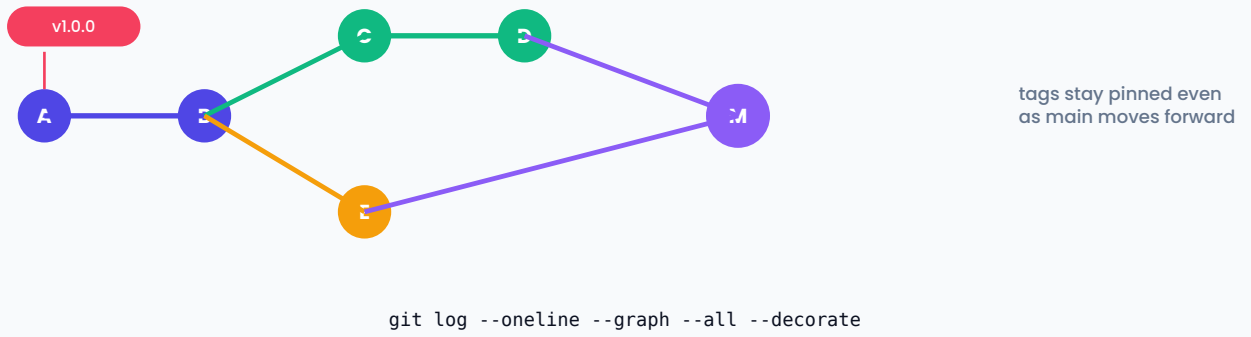


Fig 22. Branches split, work happens in parallel, then merge commit M joins them.

Inspecting previous versions

```
git show a1b2c3d                 # see what changed in a commit
git show a1b2c3d:README.md        # see a file at a past commit
git diff a1b2c3d b2c3d4e          # compare two commits
git checkout a1b2c3d               # temporarily view old code
git switch main                    # return to latest

git log --oneline --grep="login"                        # search commit messages
git log --oneline --since="2025-01-01" --until="2025-06-01"  # commits by date
```

git blame: who changed this line?

`git blame` shows, line by line, which commit and author last touched each line of a file.

```
a1b2c3d   Priya   def login():
e4f5g6h   Rohan       check_password()
a1b2c3d   Priya       return token
```

git blame file.py - who last changed each line, and in which commit

Fig 23. Every line, traced back to its commit and author.

```
git blame file.py
git blame -L 10,20 file.py   # only lines 10-20
git blame -C file.py          # also track lines moved or copied from elsewhere
```

git bisect: finding the commit that broke things

`git bisect` does a binary search through your history to find the exact commit that introduced a bug.

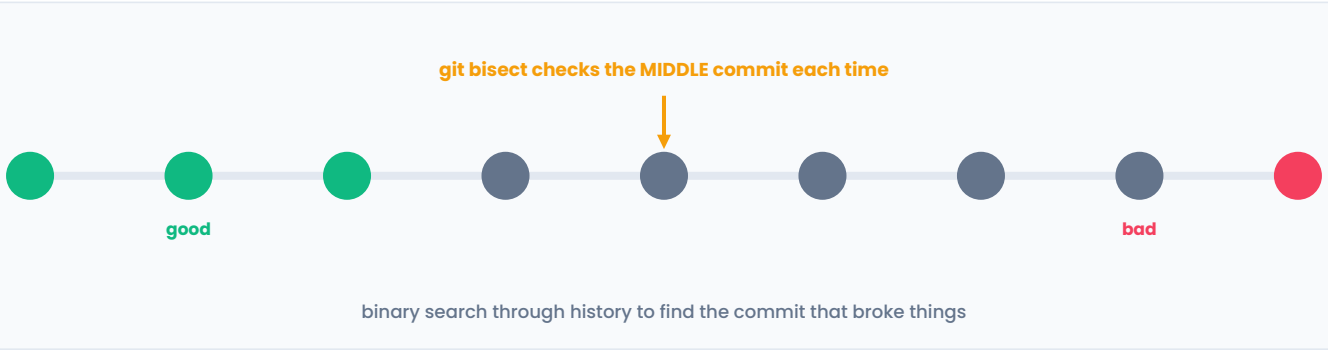


Fig 24. Mark one commit good and one bad; Git checks the middle each round.

```
git bisect start
git bisect bad                # current commit is broken
git bisect good v1.0.0         # this old commit was fine
# Git checks out a commit halfway between - test it, then:
git bisect good   # or: git bisect bad
# ... repeat until Git names the exact bad commit ...
git bisect reset              # end bisect, return to where you started

# automated version, using a test script
git bisect start HEAD v1.0.0
git bisect run npm test
```

git reflog: your safety net

If Git is a time machine, `git reflog` is its black box recorder. It remembers every place HEAD has pointed — commits, resets, rebases, checkouts — even ones `git log` cannot show, for about 90 days by default.

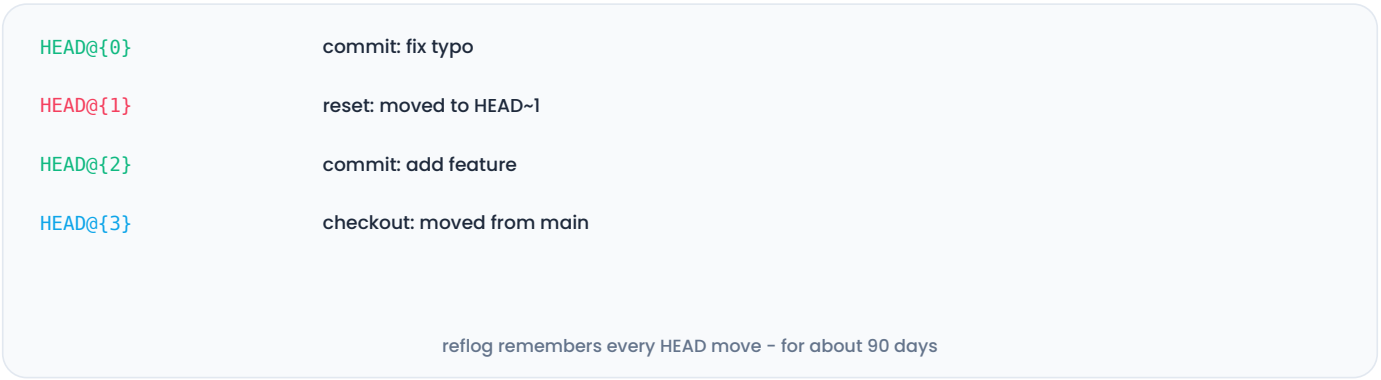


Fig 25. Every HEAD move, in order.

```
git reflog
git reflog show main

# recover a commit that reset --hard seemed to delete
git reset --hard HEAD@{2}
```

PRO TIP

Accidentally ran git reset --hard and lost work? Don't panic. Run git reflog, find the commit from before the reset, and git reset --hard back to it.

13 Git Tags

Marking releases forever

Tags mark important points in history, usually releases.

Why tags exist

- Mark version releases (v1.0.0).
- Bookmark stable checkpoints.
- Reference a specific commit permanently.

Unlike branches, tags do not move when you add new commits.

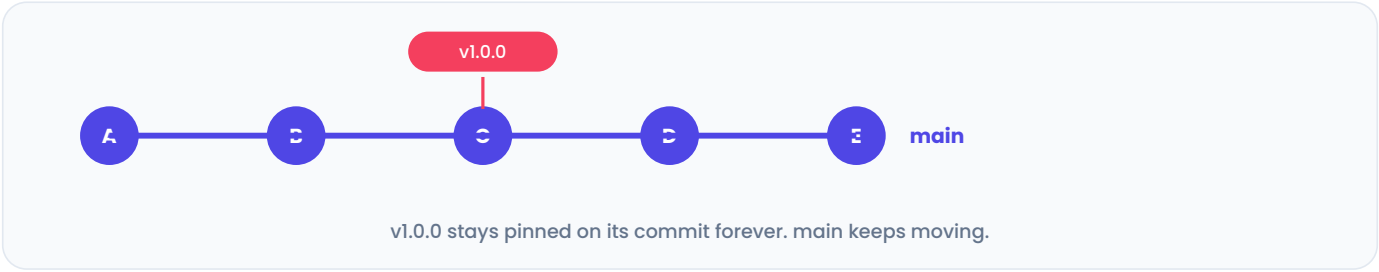


Fig 26. v1.0.0 stays on its commit forever; main keeps moving.

Lightweight vs annotated tags

Kind	What it is	Command
Lightweight	A simple pointer to a commit. No extra metadata.	`git tag v1.0.0`
Annotated	Recommended for releases. Stores author, date, and message.	`git tag -a v1.0.0 -m "msg"`

Creating tags

```
git tag v1.0.0                              # tag current commit
git tag v0.9.0 a1b2c3d                       # tag a specific commit
git tag -a v1.0.0 -m "release version 1.0.0"   # annotated, with message
```

Listing and inspecting tags

```
git tag                  # list all tags
git tag -l "v1.*"         # list tags matching a pattern
git show v1.0.0           # see tag details
```

Pushing and deleting tags

```
git push origin v1.0.0       # push one tag
git push origin --tags        # push all tags

git tag -d v1.0.0             # delete a local tag
git push origin --delete v1.0.0   # delete the remote tag
```

14 Remotes

Connecting to GitHub or any host

A **remote** is a copy of your repository hosted somewhere else — GitHub, GitLab, Bitbucket, or your own server. This section covers the Git-side commands; the GitHub notes cover the website side in depth.



Fig 27. origin is the default name for your remote.

Connecting a local repo to a remote

```
git remote add origin https://github.com/username/repo-name.git
git remote -v
```

Understanding origin and main

- `origin` is just the default NAME for your remote. You can rename it, but most people keep origin.
- `main` is the default branch name. Older repos may use `master`.

```
git remote rename origin upstream   # rename a remote
git branch -M main                   # rename current branch to main
```

First push

```
git push -u origin main
```

The `-u` flag sets main as the default upstream, so future pushes need only `git push`.

Pushing and pulling after that

```
git add .
git commit -m "update homepage"
git push

git pull origin main
```

Fetch vs pull

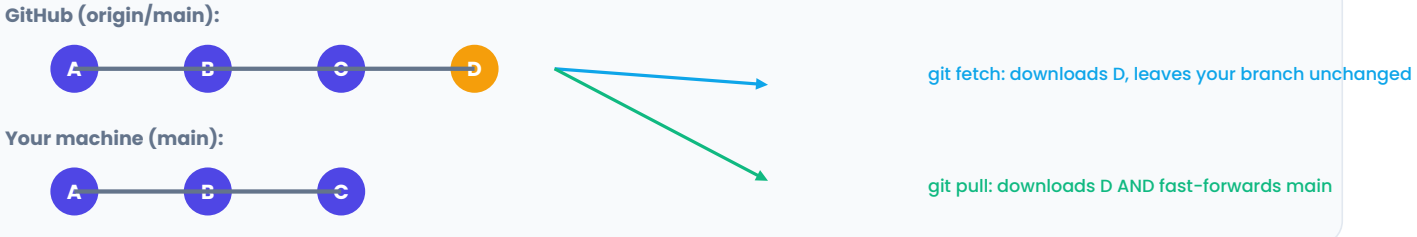


Fig 28. Fetch downloads only. Pull downloads and merges.

```
git fetch origin          # download changes WITHOUT merging
git pull origin main       # download AND merge in one step
```

Use `git fetch` when you want to review changes before merging.

Clone methods: HTTPS vs SSH



Fig 29. Both download the same thing; they differ in how you authenticate.

```
git clone https://github.com/username/repo-name.git           # HTTPS
git clone https://github.com/username/repo-name.git my-folder  # into a named folder
git clone git@github.com:username/repo-name.git                # SSH (after key setup)
```

Exploring a freshly cloned repository

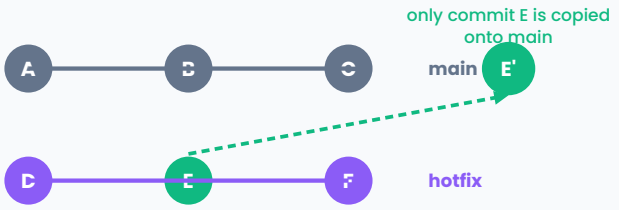
Command	What it shows
cd repo-name	Navigate into the cloned folder.
git status	Current state of the working directory.
git log --oneline -5	Last five commits, compact form.
git branch -a	All local and remote branches.
git remote -v	The fetch and push URLs for the remote.

15 Advanced Git Toolkit

cherry-pick, worktree, hooks, submodules

Tools you will reach for less often, but which solve problems nothing else does cleanly.

git cherry-pick: grab one commit from another branch



git cherry-pick <commit> - grab ONE commit from another branch

Fig 30. Copy a single commit onto your current branch.

```
git cherry-pick abc1234             # apply one commit onto the current branch
git cherry-pick abc1234..def5678     # apply a range of commits
git cherry-pick -n abc1234           # apply without committing (stage only)
git cherry-pick --continue            # after resolving a conflict
git cherry-pick --abort               # give up and go back
```

Real use: a hotfix was committed on a release branch, and you need that exact fix on main too, without merging the whole branch.

git worktree: multiple branches, checked out at once



Fig 31. One repository, several working folders.

```
git worktree add ../project-hotfix hotfix-branch   # new folder, existing branch
git worktree add -b bugfix/x ../project-fix main    # new folder AND new branch
git worktree list
git worktree remove ../project-hotfix
```

Real use: review a pull request in its own folder while your main working copy stays exactly as you left it.

Git hooks: automate actions

scripts in .git/hooks/ that run automatically at these moments

pre-commit	commit-msg	pre-push	post-merge
before a commit is made	checks the commit message	before pushing to a remote	after a merge completes

not copied by clone - share via a tracked .githooks/ folder + core.hooksPath

Fig 32. Scripts that run automatically at key moments.

```
# .git/hooks/pre-commit   (make it executable: chmod +x)
#!/bin/sh
npm run lint || exit 1    # block the commit if lint fails
```

Hooks live in `.git/hooks/` and are **not** copied by clone. To share hooks with a team, keep them in a tracked folder (e.g. `.githooks/`) and point Git at it:

```
git config core.hooksPath .githooks
```

Git submodules: nesting one repo inside another

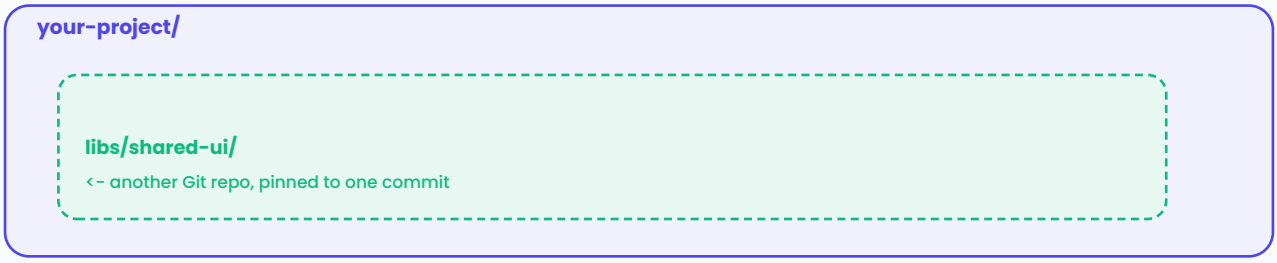


Fig 33. A submodule is pinned to one exact commit of another repo.

```
git submodule add <url> <path>              # add a submodule
git submodule init                            # initialize after cloning
git submodule update                          # fetch the pinned commit
git submodule update --init --recursive       # do both, including nested submodules
git submodule foreach git pull origin main    # update all submodules at once
```

git aliases: shortcuts for commands you type constantly

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.cm "commit -m"
git config --global alias.lg "log --oneline --graph --all"

# now:
git st        # same as git status
git lg        # same as git log --oneline --graph --all
```

git clean: deleting untracked files

```
git clean -n       # DRY RUN: show what would be deleted
git clean -f        # actually delete untracked files
git clean -fd       # also delete untracked directories
git clean -fdx      # also delete files .gitignore is hiding (use with care)
```

Other commands worth knowing

Command	What it does
git show	See the full details of one commit.
git rm	Remove a file from Git AND the disk.
git mv old new	Rename a tracked file (Git records it as a rename).
git archive	Export the repo as a .zip or .tar, without the .git folder.
.gitattributes	A file that tells Git how to treat certain paths (e.g. force LF line endings, mark a file as binary).
Git LFS	Git Large File Storage: an extension for tracking big binary files (videos, datasets) without bloating the repo.

16 Best Practices

Habits that prevent Git headaches

Good habits prevent most Git headaches.

Core commit habits

- **Commit frequently.** Small commits are easier to review, revert, and understand.
- **Keep commits small.** One logical change per commit: don't bundle unrelated fixes.
- **Write good messages.** A short summary, written in present tense.

```
git commit -m "add email field to form"
git commit -m "fix null check in user parser"
git commit -m "add dark mode toggle to settings"
```

One feature per branch

Isolate each feature in its own branch so unrelated work never mixes.

```
git switch -c feature/dark-mode
# work only on dark mode here
```

Pull before push

Always sync with the remote before pushing, to avoid unnecessary conflicts.

```
git pull origin main
git push origin main
```

Never commit secrets

No API keys, passwords, or .env files should ever enter Git history.

```
git rm --cached .env
# stops tracking the file but keeps it on your machine
echo ".env" >> .gitignore
git commit -m "remove env file from tracking"
```

WATCH OUT

If a secret was ever committed, rotate it immediately. Removing it from Git does not undo the exposure — it may still exist in old commits, forks, or cached copies.

Use .gitignore from day one

```
echo ".env" >> .gitignore
echo "__pycache__/" >> .gitignore
echo ".DS_Store" >> .gitignore
git add .gitignore
git commit -m "add gitignore"
```

Review before every commit

1. Check what's changed: `git status`
2. Review the actual edits: `git diff` and `git diff --staged`
3. Commit with a clear message: `git commit -m "..."`

Using Git in the AI era

AI tools write code fast, but they can also introduce bugs or overwrite working code. Git lets you review, commit, and roll back AI output exactly like any other change.

1. Commit before letting AI make large changes: `git add . && git commit -m "working state before AI refactor"`
2. Review every line AI changed: `git diff`
3. Roll back if the result is bad: `git restore .` or `git reset --hard HEAD~1`

> **KEY IDEA**
> Never commit AI output blindly. Some tools suggest commit messages — edit them to be specific rather than generic: "update files" tells you nothing a year from now; "add input validation to signup form" does.

Quick reference: everyday habits

Command	When to use it
git status	Check what's changed before staging.
git diff --staged	Review exactly what a commit will contain.
git rm --cached file	Stop tracking a file without deleting it locally.
git switch -c name	Create an isolated branch for one feature.
git pull origin main	Sync before you start work, and before you push.

17 Cheat Sheet & Glossary

Every command, one place

Quick copy-paste reference for every command in this guide.

Setup & Config

`git config --global user.name "N"`	Set global username
`git config --global user.email "e"`	Set global email
`git config --list`	List all config
`git --version`	Check installed version

Repository Basics

`git init`	Starts a new repository
`git clone <url>`	Copies a remote repository
`git status`	Checks the current state
`git status -s`	Short, compact status

Tracking Changes

`git add <file>`	Stages one file
`git add .`	Stages all changes
`git add -p`	Stage interactively, hunk by hunk
`git commit -m "msg"`	Saves a snapshot
`git commit -am "msg"`	Stage tracked files + commit

Branches

`git branch`	Lists local branches
`git branch -a`	Lists all branches
`git switch <b>`	Switches to a branch
`git switch -c <b>`	Creates + switches
`git merge <b>`	Merges a branch in
`git branch -d <b>`	Deletes a local branch

Remotes

`git remote -v`	Shows configured remotes
`git remote add origin <url>`	Connects repo to a host
`git push -u origin main`	First push, sets upstream
`git push`	Push using tracked upstream
`git pull origin main`	Pull + merge remote changes
`git fetch origin`	Download without merging

History

`git log`	Full commit history
`git log --oneline`	Compact history
`git log --oneline --graph --all`	Visual graph
`git show <commit>`	Inspect one commit
`git diff`	Unstaged changes
`git diff --staged`	Staged changes

Undo

`git restore <file>`	Discard unstaged edits
`git restore --staged <file>`	Unstage (keep edits)
`git reset --soft HEAD~1`	Undo commit, keep staged
`git reset --hard HEAD~1`	Undo commit, discard all
`git revert <commit>`	Safely undo a pushed commit

Stash

`git stash`	Save uncommitted work
`git stash push -m "msg"`	Stash with a label
`git stash list`	List stashes
`git stash pop`	Apply + remove latest stash
`git stash drop`	Remove latest stash

Tags

`git tag v1.0.0`	Create a lightweight tag
`git tag -a v1.0.0 -m "msg"`	Create an annotated tag
`git push origin --tags`	Push all tags
`git tag -d v1.0.0`	Delete a local tag

Rebase

`git rebase main`	Replay commits on top of main
`git rebase -i HEAD~n`	Interactive rebase, last n commits
`git rebase --abort`	Cancel an in-progress rebase
`git push --force-with-lease`	Push rewritten history, safely

Inspection	
`git blame file`	Who last changed each line
`git bisect start`	Begin a binary-search bug hunt
`git reflog`	See every HEAD move (90-day safety net)

Advanced Toolkit	
`git cherry-pick <sha>`	Copy one commit to this branch
`git worktree add <path> <b>`	Check out a 2nd branch in parallel
`git submodule update --init`	Fetch pinned sub-repos
`git clean -fd`	Delete untracked files + folders

Mini glossary

Term	Meaning
`Repository (repo)`	A project folder tracked by Git, including its .git history.
`Commit`	A saved snapshot of the project at one moment, with a unique SHA hash.
`Branch`	An independent, movable line of development.
`HEAD`	A pointer to your current commit / branch tip.
`Working directory`	Your files on disk, as you edit them.
`Staging area (index)`	A holding zone for changes about to be committed.
`Remote`	A copy of the repository hosted elsewhere (e.g. GitHub).
`origin`	The default nickname for your main remote.
`Upstream`	The remote branch your local branch tracks and compares against.
`Merge`	Combining the history of two branches, keeping both.
`Fast-forward`	A merge with no new commit, because history is already linear.
`Rebase`	Replaying commits on top of another branch for a linear history.
`Conflict`	When Git cannot auto-merge two changes to the same lines.
`Stash`	A shelf for uncommitted work you want to set aside.
`Tag`	A permanent, non-moving pointer to one commit, usually a release.
`SHA / hash`	The unique ID of a commit (e.g. a1b2c3d).
`Fork`	Your own copy of someone else's repo, on the host.
`Clone`	A local copy of a repo, downloaded to your machine.
`.gitignore`	A file listing paths Git should never track.
`Detached HEAD`	HEAD points directly at a commit instead of a branch; new commits here can get lost unless you create a branch.

TRY IT

Practice loop: make a folder, run git init, write a file, git add . && git commit -m "first commit", then try branching, merging, stashing and reverting on your own small project until each command feels automatic.

That's Git. The GitHub notes pick up from here: pushing to a real remote, pull requests, GitHub Actions, GitHub Desktop, and the full gh CLI reference.